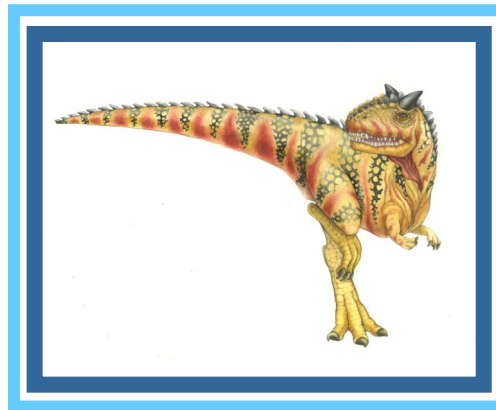


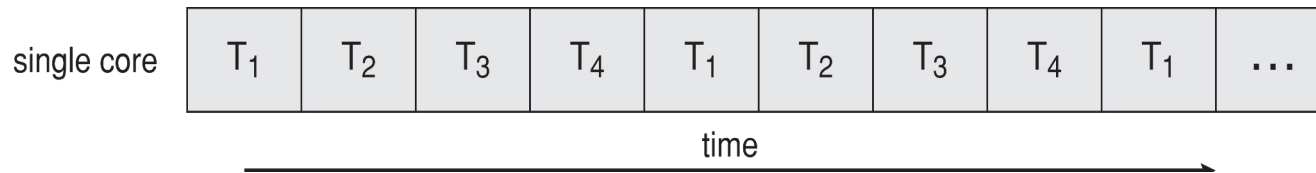
Chapter 6: CPU Scheduling





Scheduling Criteria

- ❑ **CPU utilization** – keep the CPU as busy as possible
- ❑ **Throughput** – # of processes that complete their execution per time unit



- ❑ **Turnaround time** – amount of time to execute a particular process
- ❑ **Waiting time** – amount of time a process has been waiting in the ready queue
- ❑ **Total Turnaround time = Burst time + Waiting time**





Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum** q), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

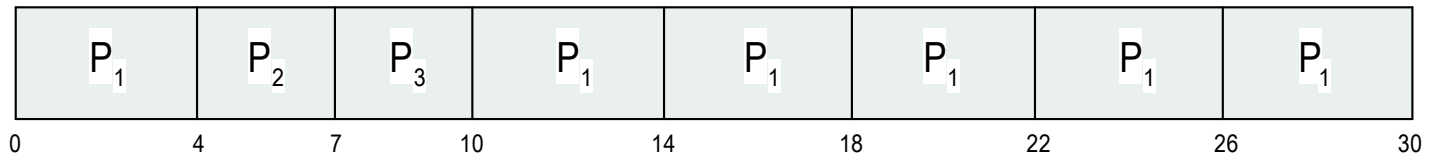




Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

□ The Gantt chart is:



□ Typically, higher average turnaround than SJF, but better **response**

□ What is the average waiting time when $q = 4$?

□ q should be large compared to context switch time **17/3**

□ q usually 10ms to 100ms, context switch < 10 usec





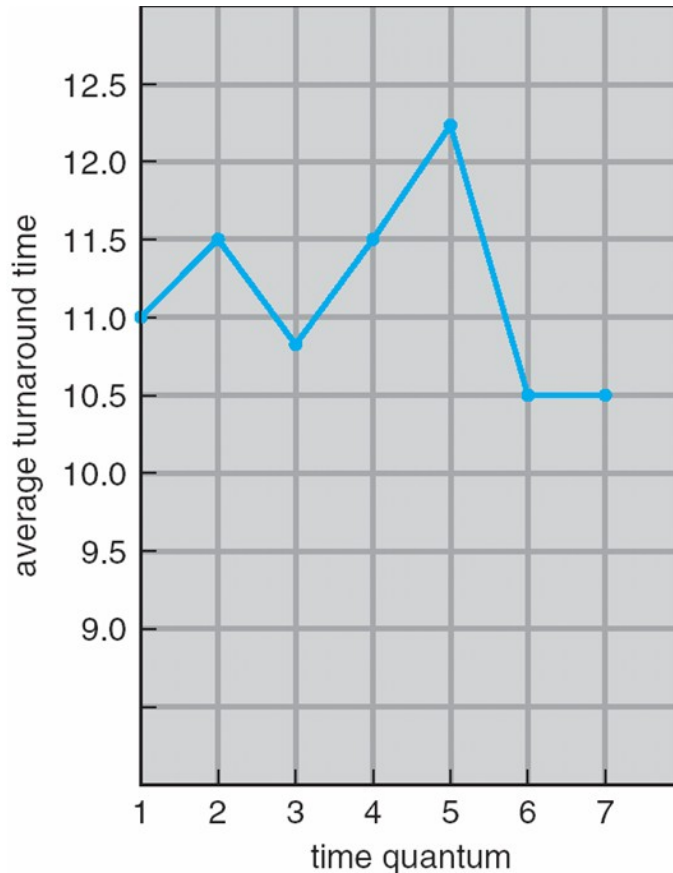
Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum** q), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1)q$ time units.
- Timer interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high





Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7

80% of CPU bursts
should be shorter than q





Algorithm Evaluation

- How to select CPU-scheduling algorithm for an OS?
- Determine criteria, then evaluate algorithms
- Deterministic modeling**
 - Type of **analytic evaluation**
 - Takes a particular predetermined workload and defines the performance of each algorithm for that workload
- Consider 5 processes arriving at time 0:

<u>Process</u>	<u>Burst Time</u>
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

- What is the ~~average waiting time~~ performance of FCFS, SJF and RR?



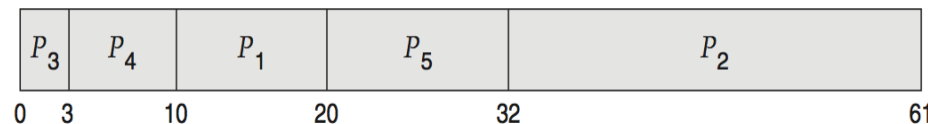


Deterministic Evaluation

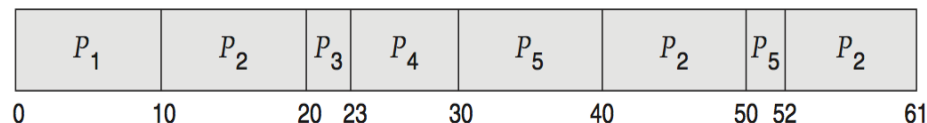
- For each algorithm, calculate minimum average waiting time
- Simple and fast, but requires exact numbers for input, applies only to those inputs
- FCS is 28 ms:



- Non-preemptive SJF is 13 ms:



- RR (with $\alpha=10$) is 23 ms:





Discussion

- FCFS
- SJF
- RR

- *Any other algorithm that you can develop to further minimize the turnaround time or maximize the throughput?*





Suppose the CPU scheduler is given 4 tasks, A, B, C and D. Each task requires a certain number of time units to complete using round-robin algorithm.

Task	CPU Time(msec)
A	8
B	4
C	9
D	5

If time quanta = 6,

- (i) What is the average turn-around time?
- (ii) What is the average wait time?

Answers:

- I. 21, 13.5
- II. 20, 13.5
- III. 20, 13
- IV. 21, 13
- V. None of the above

II





Process scheduling: The information about 4 processes is given in the following table.

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
<i>P1</i>	0.0	7
<i>P2</i>	2.0	4
<i>P3</i>	4.0	1
<i>P4</i>	5.0	4

If the operating system is designed to use preemptive SJF (Shortest job first) algorithm for its process scheduling, compute: the average waiting time and the average turnaround time?

Answers:

- I. 3.5, 7
- II. 4, 12
- III. 4, 7
- IV. 3, 7
- V. None of the above

IV





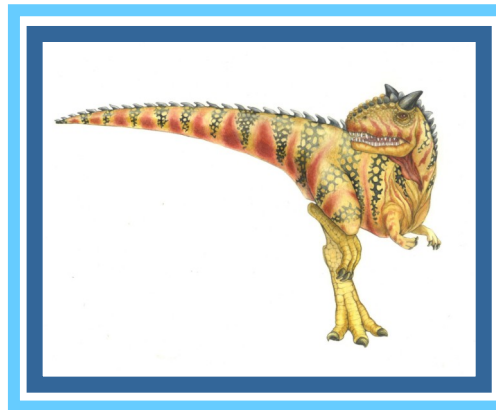
Process

- Process concept, state
- Process creation, deletion, management
- Process communications
- Process synchronization
- Deadlock, and deadlock avoidance
- Process scheduling

.....



End of Chapter 6





Tips for Problem 3 in Project 2

- ❑ Consider each passenger as a thread
- ❑ Consider each handler, screener and attendant as a thread
- ❑ The handlers, screeners and attendants are serving passengers, so they are in the waiting state at the beginning, and will be signaled by the staff in the previous step.
- ❑ For example:

```
void baggageHandler() {
    sem_wait(&baggage);
    // ... a series of operations
    sem_post(&security);
}

void securityScreener() {
    sem_wait(&security);
    // ... a series of operations
    sem_post(&attendant);
}

..... • •
```

